PayShield — техническая документация

Проект: учебный платёжный сервис для клиники «Мед-Сервис»

Кейс: № 1 «Платёжный сервис: безопасный платёж за услугу»

Версия документа: 1.0.0 (финальная базовая версия)

Дата подготовки: 9 ноября 2026 г.

Реализация: Python CLI-прототип

API: OpenAPI 3.0.3 — целевой HTTP-контракт

> **Статус и границы документа.** Документ описывает исправленную версию CLI-прототипа и связанные с ней проектные спецификации. Сам прототип не является HTTP-сервером: он читает JSON-файл, проверяет данные, вызывает локальную функцию эмулятора, записывает результат в CSV и выводит JSON в консоль. Методы OpenAPI, двухэтапное подтверждение по токену, SMS и биометрическая заглушка описывают целевую архитектуру и пока не реализованы в CLI. Эти два уровня не следует смешивать при защите проекта.

Содержание

1. Назначение и область применения
2. Архитектура и компоненты
3. Структура проекта и окружение
4. Требования к входным данным
5. Правила валидации
6. Жизненный цикл и статусы платежа
7. Идемпотентность и защита от дублей
8. Эмулятор платёжного шлюза
9. Хранение данных и CSV-схемы
10. Формат ответов и каталог ошибок
11. API-контракт OpenAPI
12. Подтверждение платежа: целевая спецификация
13. Журналирование и трассировка
14. Безопасность и ограничения
15. Установка, запуск и эксплуатация
16. Тестирование и результаты
17. Матрица реализации и дальнейшие работы
18. Состав документации и источники

---

1. Назначение и область применения

PayShield — учебный прототип встраиваемого платёжного сервиса для клиники «Мед-Сервис». Предполагаемый внешний клиент — приложение клиники, через которое пациент выбирает услугу и оплачивает её. Платёжный сервис принимает параметры заказа, выполняет форматно-логические проверки, имитирует взаимодействие с платёжным шлюзом и возвращает результат.

В рамках учебного кейса реальный эквайринг заменён детерминированным эмулятором. Сервис не выполняет реальное списание денег, не подключается к банковским системам и не принимает реальные данные банковских карт.

1.1. Цели решения

- определить структуру данных платежа и получателя;
- формализовать правила валидации входных данных;
- описать API создания, подтверждения, получения и отмены платежа;
- обеспечить защиту от повторной обработки запроса;
- смоделировать успешную оплату, отказ и таймаут;
- сохранять результаты операций и события для последующего анализа;
- подтвердить поведение системы тестовыми сценариями.

1.2. Что реализовано в текущем прототипе

Прототип запускается из командной строки. На один запуск ему передаётся один JSON-файл. Программа загружает справочник активных получателей, читает накопленные платежи, валидирует запрос, проверяет идемпотентность и дубликат заказа, затем выбирает результат эмулятора. Результат записывается в `payments_out.csv`, первоначальный JSON-ответ сохраняется в поле `response_json`, события записываются в `payment_service.log` и `gateway.log`.

1.3. Что не входит в текущую реализацию

- работающий HTTP-сервер и маршруты `/api/v1/...`;
- реальное обращение к банку или платёжному провайдеру;
- создание и проверка `confirm_token`;
- отправка SMS и реальное биометрическое подтверждение;
- отдельные исполняемые операции подтверждения, получения статуса и отмены;
- постоянная база данных вместо CSV-файлов;
- процедура сверки окончательного результата платежа после таймаута.

---

2. Архитектура и компоненты

2.1. Логическая схема



Схема показывает логические компоненты проекта. В текущем коде эмулятор — функция Python `emulate_gateway()`, а не отдельный сетевой процесс. Клиентом при проверке служит JSON-файл, передаваемый при запуске скрипта.

2.2. Ответственность компонентов

Компонент	Назначение	Реализация
Клиент клиники	Передаёт параметры оплаты и получает результат	JSON-файл для CLI; предполагаемое внешнее приложение для API
PayShield	Проверяет вход, применяет правила повторов, формирует результат	`05_app.py`
Справочник получателей	Хранит идентификаторы и статусы получателей	`01_данные/recipients.csv`
Эмулятор шлюза	Возвращает воспроизводимый результат операции	`emulate_gateway()` в `05_app.py`
Хранилище платежей	Сохраняет платежи и исходные ответы	`payments_out.csv`

Компонент	Назначение	Реализация
Журнал сервиса	Фиксирует ошибки и ключевые этапы	`payment_service.log`
Журнал эмулятора	Фиксирует выбранный сценарий шлюза	`gateway.log`

2.3. Основной поток выполнения

19. Скрипт получает путь к JSON-файлу из аргументов командной строки.
20. Загружает и разбирает JSON; корнем должен быть объект.
21. Загружает список активных получателей из `recipients.csv`.
22. Загружает существующие платежи из `payments_out.csv`; при необходимости выполняет миграцию заголовка с 12 до 13 колонок.
23. Выполняет проверки V-1…V-7.
24. Ищет активный ключ идемпотентности: совпадающие данные дают повтор исходного ответа, отличающиеся — конфликт.
25. Проверяет дубликат пары `order_id + recipient`.
26. Вызывает детерминированный эмулятор.
27. Формирует статус и код ошибки, генерирует `payment_id` и `correlation_id`.
28. Сохраняет платёж и исходный ответ; пишет журналы; выводит JSON в консоль.

---

3. Структура проекта и окружение

3.1. Основные файлы

Путь	Содержание
`05_app.py`	CLI-прототип платёжного сервиса
`02_правила.md`	Правила валидации и обработки V-1…V-9
`03_data_dictionary.md`	Словарь полей исходных и выходных данных
`04_requirements.md`	Функциональные и нефункциональные требования
`06_openapi.yaml`	Целевой HTTP-контракт OpenAPI 3.0.3
`07_confirmation.md`	Спецификация методов подтверждения
`08_gateway_emulator.md`	Правила детерминированного эмулятора
`test.md`	Пять обязательных сценариев T1-1…T1-5
`tests/test_prototype.py`	Автоматизированные интеграционные тесты CLI
`01_данные/payments.csv`	Исходный набор платежей, 12 колонок
`01_данные/recipients.csv`	Справочник получателей
`payments_out.csv`	Рабочий журнал новых платежей, 13 колонок после миграции

3.2. Требования к окружению

- Python 3.10 или новее;
- стандартная библиотека Python: `csv`, `hashlib`, `json`, `logging`, `re`, `sys`, `uuid`, `datetime`, `decimal`, `pathlib`, `typing`;
- сторонние пакеты для запуска самого прототипа не требуются;
- macOS, Linux или Windows с установленным Python 3.10+.

3.3. Расположение справочника

Код ищет файл получателей в следующем порядке:

29. `01_данные/recipients.csv` относительно расположения `05_app.py`;
30. `recipients.csv` рядом с `05_app.py`.

Файл читается с кодировкой `cp1251` (Windows-1251). Активными считаются только строки, у которых поле `status` после удаления пробелов и приведения к верхнему регистру равно `ACTIVE`.

4. Требования к входным данным

4.1. JSON-запрос CLI

Пример запроса:

```
{
  "order_id": "ord_1001",
  "recipient": "clinic_medservice",
  "purpose": "Приём терапевта",
  "amount": 2500.00,
  "currency": "RUB",
  "idempotency_key": "idem_t1_01"
}
```

В CLI ключ передаётся JSON-полем `idempotency_key`. В целевом HTTP API тот же смысловой ключ передаётся заголовком `Idempotency-Key`.

4.2. Обязательные поля

Поле	Тип	Обязательность	Ограничение
`order_id`	string	Да	`^ord_[0-9]{4}$`
`recipient`	string	Да	Существующий ID со статусом `ACTIVE`
`purpose`	string	Да	1–200 символов; проверка на PAN/CVV
`amount`	number или десятичная строка	Да	> 0; ≤ 1 000 000; до 2 знаков после точки
`currency`	string	Да	Только `RUB`
`idempotency_key`	string	Да для CLI	8–64 символа; TTL 24 часа

4.3. Идентификатор заказа

Формат: `ord_` и ровно четыре цифры, например `ord_1001`. Проверка выполняется регулярным выражением `^ord_[0-9]{4}$`.

4.4. Сумма

Сумма обрабатывается через `Decimal`, чтобы избежать типичных ошибок двоичной арифметики `float`. Допустимый диапазон — строго больше нуля и не более 1 000 000. Допускается не более двух цифр после десятичной точки. На входе используется обычная десятичная запись; научная нотация не соответствует регулярному выражению `AMOUNT_PATTERN`.

4.5. Назначение платежа

Длина — от 1 до 200 символов. Проверяется наличие последовательности из 13–19 цифр, похожей на PAN. CVV/CVC выявляется по явной текстовой метке `CVV` или `CVC` рядом с 3–4 цифрами. Произвольную последовательность из 3–4 цифр невозможно достоверно классифицировать как CVV без контекста.

5. Правила валидации

ID	Условие	Результат проверки	Код
V-1	Обязательное поле отсутствует или пустое	Запрос отклоняется до вызова эмулятора	`VALIDATION_ERROR`
V-2	`order_id` не соответствует `^ord_[0-9]{4}$`	Запрос отклоняется	`INVALID_FORMAT`
V-3	`amount` ≤ 0 или > 1 000 000	Запрос отклоняется	`AMOUNT_OUT_OF_RANGE`

ID	Условие	Результат проверки	Код
V-4	`currency` не равно `RUB`	Запрос отклоняется	`UNSUPPORTED_CURRENCY`
V-5	Получатель не найден или не `ACTIVE`	Запрос отклоняется	`RECIPIENT_NOT_FOUND`
V-6	`purpose` пустое/длиннее 200 символов/содержит PAN или помеченный CVV/CVC	Запрос отклоняется	`INVALID_PURPOSE`
V-7	Сумма нечисловая либо содержит более двух знаков после точки	Запрос отклоняется	`AMOUNT_TYPE_ERROR` / `INVALID_AMOUNT_PRECISION`
V-8	Уже существует пара `order_id + recipient`	Новый платёж не создаётся; возвращается конфликт с информацией о существующей записи	`DUPLICATE_ORDER`
V-9	Повтор активного `Idempotency-Key` с теми же параметрами	Возвращается сохранённый результат с `replayed: true`, без нового вызова шлюза	Повтор, HTTP-маппинг 200

5.1. Дополнительные технические ошибки

- `INVALID_REQUEST` — файл не читается, JSON некорректен или корень JSON не является объектом.
- `IDEMPOTENCY_KEY_CONFLICT` — тот же ключ в течение TTL используется с другими данными.
- `DUPLICATE_ORDER` — повтор заказа по паре `order_id + recipient` с новым ключом.
- `GATEWAY_TIMEOUT` — эмулятор не вернул своевременный результат; платёж остаётся `PENDING`.

5.2. Порядок проверок

Валидация обязательных и форматных полей выполняется до проверок повторов. Далее порядок следующий:

31. `check_idempotency()`;
32. `find_duplicate_order()`;
33. `emulate_gateway()`.

Такой порядок важен: точный повтор ранее выполненного запроса должен получить исходный результат, а не быть ошибочно определён как повторный заказ.

---

6. Жизненный цикл и статусы платежа

6.1. Статусы

Статус	Значение	В текущем CLI
`CREATED`	Платёж создан и ожидает подтверждения	Отдельный этап отсутствует
`PENDING`	Результат шлюза неизвестен из-за таймаута	Реализован
`SUCCESS`	Эмулятор подтвердил успешную операцию	Реализован
`DECLINED`	Эмулятор отклонил операцию	Реализован
`ERROR`	Техническая ошибка обработки	Используется для ответа об ошибке запроса; запись платежа не создаётся
`CANCELED`	Платёж отменён до подтверждения	Не реализован

`TIMEOUT` — это результат сценария эмулятора, а не самостоятельный статус платежа. В журнале платежей сохраняется `PENDING` и код `GATEWAY_TIMEOUT`.

6.2. Переходы целевой модели

- `CREATED → PENDING` — клиент инициировал подтверждение, операция передана шлюзу;
- `PENDING → SUCCESS` — шлюз подтвердил успешную обработку;
- `PENDING → DECLINED` — шлюз отклонил обработку;

- `PENDING → PENDING` — таймаут, результат остаётся неизвестным;
- `CREATED → CANCELED` — клиент отменил платёж до подтверждения.

Текущая CLI-реализация объединяет создание и обработку эмулятором в одном запуске. Поэтому `CREATED` и `CANCELED` отражены в целевой модели и OpenAPI, но не исполняются скриптом.

---

7. Идемпотентность и защита от дублей

7.1. Назначение

Идемпотентность нужна для безопасной обработки повторной доставки запроса. Например, клиент может повторить отправку из-за задержки сети. Повтор не должен запускать второй платёж.

7.2. Параметры ключа

- поле CLI: `idempotency_key`;
- заголовок целевого API: `Idempotency-Key`;
- допустимая длина: 8–64 символа;
- срок действия: 24 часа от `created_at` сохранённой записи.

7.3. Алгоритм

При совпадении ключа проверяются поля `order_id`, `recipient`, `purpose`, `amount`, `currency`.

Состояние	Условие	Действие
`NEW`	Активный ключ не найден	Продолжить обработку
`REPLAY`	Ключ и все параметры совпадают	Вернуть сохранённый результат платежа, добавить `replayed: true` и HTTP-маппинг 200; не вызывать шлюз
`CONFLICT`	Ключ совпал, хотя бы одно бизнес-поле отличается	Вернуть `IDEMPOTENCY_KEY_CONFLICT`, HTTP-маппинг 409

В `payments_out.csv` сохраняется поле `response_json` с первоначальным результатом операции. При повторе он возвращается с `replayed: true` и HTTP-маппингом 200. Для старых записей без этого поля используется резервное формирование ответа из сохранённых колонок платежа.

7.4. Дубли заказа — отдельная проверка

V-8 проверяет пару `order_id + recipient`. Если она уже присутствует в журнале, новый платёж не создаётся и возвращается `DUPLICATE_ORDER` с HTTP-маппингом 409. Проверка идемпотентности выполняется раньше: точный повтор с прежним ключом возвращает replay.

7.5. Особенности TTL и старых записей

Если в старой записи нет `created_at` или дата не разбирается, `is_key_active()` считает ключ активным для совместимости с данными. Нормальные записи сравниваются с текущим UTC-временем. По истечении 24 часов ключ больше не участвует в поиске активного повтора.

---

8. Эмулятор платёжного шлюза

8.1. Детерминизм

Эмулятор вычисляет:

```
bucket = int(hashlib.md5(order_id.encode("utf-8")).hexdigest(), 16) % 10
```

Одинаковый `order_id` всегда даёт одинаковый bucket и одинаковый сценарий. MD5 используется только как детерминированный способ выбора учебной ветки, а не как криптографическая защита.

8.2. Таблица сценариев

bucket	Результат функции	Статус платежа	Код ошибки
0–6	SUCCESS	SUCCESS	отсутствует
7	DECLINED	DECLINED	INSUFFICIENT_FUNDS
8	DECLINED	DECLINED	CARD_DECLINED
9	TIMEOUT	PENDING	GATEWAY_TIMEOUT

8.3. Контрольные идентификаторы

order_id	bucket	Ожидаемый результат
ord_1001	1	SUCCESS
ord_1002	7	DECLINED / INSUFFICIENT_FUNDS
ord_1007	8	DECLINED / CARD_DECLINED
ord_1018	9	TIMEOUT / PENDING

8.4. Таймаут

При таймауте нельзя утверждать, что платёж точно не прошёл: сервис не получил окончательного ответа. Поэтому CLI сохраняет PENDING и GATEWAY_TIMEOUT. Повтор с тем же ключом возвращает сохранённый PENDING; повторный вызов эмулятора не выполняется. Получение окончательного статуса после таймаута остаётся отдельной задачей целевого API.

---

9. Хранение данных и CSV-схемы

9.1. Исходный payments.csv

Исходный файл содержит 12 полей:

34. payment_id;
35. order_id;
36. recipient;
37. purpose;
38. amount;
39. currency;
40. status;
41. error_code;
42. idempotency_key;
43. created_at;
44. confirmed_at;
45. correlation_id.

Пример существующей записи из исходного набора: pay_0001, ord_1001, clinic_medservice, Приём терапевта, 2500.00, RUB, SUCCESS, ключ idem_0001, время создания 2025-11-01T08:00:00Z, подтверждение 2025-11-01T08:00:03Z, correlation ID corr_0001.

9.2. Выходной payments_out.csv

Выходной журнал содержит исходные 12 колонок и дополнительную колонку response_json. Она используется для точного воспроизведения первоначального ответа при идемпотентном повторе. Исходный payments.csv не изменяется.

9.3. Мягкая миграция схемы

Если `payments_out.csv` имеет старую 12-колоночную схему, загрузчик переписывает выходной журнал через временный файл и добавляет `response_json`. Старые записи сохраняются, новое поле для них остаётся пустым. При повторе таких записей код строит резервный ответ по имеющимся полям.

9.4. Генерация серверных полей

- `payment_id`: `pay_demo_` + последние четыре цифры `order_id`;
- `created_at`: текущее UTC-время в ISO 8601;
- `confirmed_at`: время для `SUCCESS` и `DECLINED`; пусто для `PENDING`;
- `correlation_id`: `corr_` + 12 шестнадцатеричных символов из UUID;
- `response_json`: JSON первоначального ответа.

---

10. Формат ответов и каталог ошибок

10.1. Успех

```
{
  "payment_id": "pay_demo_1001",
  "status": "SUCCESS",
  "error_code": null,
  "http_status": 200,
  "retryable": false,
  "correlation_id": "corr_example",
  "payment": {
    "payment_id": "pay_demo_1001",
    "order_id": "ord_1001",
    "recipient": "clinic_medservice",
    "status": "SUCCESS"
  }
}
```

10.2. Ошибка валидации

В ответе указываются основной код ошибки, сообщение, `retryable: false`, `correlation_id` и массив `details` с полями и конкретными ошибками. Для `RECIPIENT_NOT_FOUND` CLI формирует `http_status: 404`; остальные ошибки форматно-логического контроля возвращают 422.

10.3. Таблица ошибок

Код	Значение	HTTP-маппинг	Повтор
`VALIDATION_ERROR`	Обязательное поле отсутствует	422	Нет
`INVALID_FORMAT`	Неверный формат идентификатора или ключа	422	Нет
`AMOUNT_TYPE_ERROR`	Сумма не является числом	422	Нет
`AMOUNT_OUT_OF_RANGE`	Сумма вне допустимого диапазона	422	Нет
`INVALID_AMOUNT_PRECISION`	Более двух десятичных знаков	422	Нет
`UNSUPPORTED_CURRENCY`	Валюта не поддерживается	422	Нет
`RECIPIENT_NOT_FOUND`	Получатель отсутствует или неактивен	404	Нет
`INVALID_PURPOSE`	Назначение недопустимо	422	Нет
`DUPLICATE_ORDER`	Такой заказ с получателем уже записан	409	Нет
`IDEMPOTENCY_KEY_CONFLICT`	Тот же ключ с другими параметрами	409	Нет
`INSUFFICIENT_FUNDS`	Эмулятор имитировал недостаток средств	402	Нет
`CARD_DECLINED`	Эмулятор отклонил операцию	402	Нет

Код	Значение	HTTP-маппинг	Повтор
GATEWAY_TIMEOUT	Эмулятор не ответил вовремя	504; статус PENDING	Только безопасная проверка статуса
INVALID_REQUEST	Входной JSON нельзя обработать	400	Нет

10.4. HTTP-код и код завершения CLI — разные понятия

Поле `http_status` в JSON — это отображение предполагаемого HTTP-кода для результата. Оно не означает, что CLI действительно отправил HTTP-ответ. Код завершения процесса сообщает оболочке результат выполнения:

Код выхода	Смысл
0	Успех или безопасный replay
1	Отказ эмулятора, конфликт ключа, дубликат заказа или ошибка аргументов
2	Ошибка входного файла/JSON или валидации
3	Таймаут эмулятора; платёж сохранён как PENDING

---

11. API-контракт OpenAPI

Файл `06_openapi.yaml` описывает целевой HTTP API версии `3.0.3`. Он успешно разбирается YAML-парсером. Наличие описания маршрута не означает, что маршрут поднят в текущем Python CLI.

11.1. Авторизация

В контракте предусмотрена схема `bearerAuth` — HTTP Bearer. Текущий CLI не проверяет токены авторизации.

11.2. Методы

Метод и путь	Назначение	Основные ответы
`GET /health`	Проверка доступности	200
`POST /api/v1/payments`	Создание платежа со статусом CREATED	201, 400, 401, 404, 409, 422
`POST /api/v1/payments/{id}/confirm`	Подтверждение и обработка платежа	200, 401, 402, 404, 409, 410, 422, 504
`GET /api/v1/payments/{id}`	Получение статуса	200, 401, 404
`POST /api/v1/payments/{id}/cancel`	Отмена до подтверждения	200, 401, 404, 409

11.3. Создание платежа

Тело `PaymentCreate` содержит `order_id`, `recipient`, `purpose`, `amount`, `currency`. Ключ передаётся обязательным заголовком `Idempotency-Key` длиной 8–64 символа. Успешное создание возвращает HTTP 201 и `PaymentCreated`, включающий `payment_id`, `status: CREATED`, `confirm_token`, `expires_in: 600`.

11.4. Подтверждение

`POST /api/v1/payments/{id}/confirm` принимает метод подтверждения и `confirm_token`. Код 200 означает успешный результат; 402 — отказ эмулятора; 410 — истёкший или использованный токен; 422 — неправильные данные подтверждения; 504 — шлюз не ответил вовремя и платёж остаётся PENDING.

11.5. Разделение API и CLI

В целевом API создание платежа и его подтверждение — отдельные вызовы. В CLI-прототипе они объединены: программа сразу проводит запрос через локальную функцию эмулятора. Поэтому финальный CLI-ответ об успехе может содержать `http_status: 200`, тогда как целевой HTTP-вызов создания возвращает `201` на стадии `CREATED`.

---

12. Подтверждение платежа: целевая спецификация

12.1. Одноразовый токен

- время действия: 10 минут (600 секунд);
- токен используется только один раз;
- сервер хранит только хеш токена;
- токен связан с `payment_id`, сроком действия и признаком использования;
- истёкший или использованный токен не запускает обращение к шлюзу.

12.2. Методы подтверждения

Метод	Назначение	Ограничение
`card_token`	Имитация подтверждения картой по токену	Реальные данные карты не принимаются
`sms_code`	Имитация подтверждения одноразовым SMS-кодом	До трёх попыток; отправка SMS не обязательна для учебного прототипа
`biometric_stub`	Заглушка биометрического подтверждения	Реальное сканирование не производится

12.3. Зафиксированная спецификация версии 1.0.0

Для единообразия документации целевое поведение фиксируется следующим образом:

- истёкший или уже использованный `confirm_token` даёт HTTP 410; шлюз не вызывается, платёж остаётся `CREATED`;
- после трёх неверных SMS-кодов текущая попытка подтверждения блокируется; платёж не переводится автоматически в `DECLINED`, поскольку банк не обрабатывал операцию;
- повторная выдача токена в версии 1.0.0 не предоставляется;
- доступ к операциям над платежом в целевом API требует Bearer-аутентификации; проверка владельца платежа должна выполняться серверным слоем авторизации;
- после таймаута платёж остаётся `PENDING`; клиент может получить статус отдельным `GET`, но повторное списание автоматически не запускается.

Это зафиксированные правила целевой спецификации. В текущем CLI нет генерации/проверки токенов, SMS, Bearer-аутентификации и HTTP-маршрутов; поэтому перечисленные правила не являются утверждением о реализованном поведении программы.

---

13. Журналирование и трассировка

13.1. `payment_service.log`

Сервисный журнал создаётся рядом со скриптом. Используется уровень `INFO` и выше, формат содержит время, уровень и сообщение. В нём фиксируются ошибки чтения входных данных, ошибки валидации, конфликт идемпотентности, replay, дубликат заказа и итог обработки платежа.

13.2. `gateway.log`

Файл содержит отдельные JSON-строки. Структура события:

```
{
  "timestamp": "2026-11-09T12:00:00Z",
  "order_id": "ord_1001",
  "correlation_id": "corr_example",
  "result": "SUCCESS",
  "error_code": null,
  "duration_ms": 0
}
```

`duration_ms` в текущей реализации устанавливается в `0` константой; это не измерение реального времени ожидания. Поля предназначены для единого формата аудита в будущем.

13.3. Корреляция событий

`correlation_id` создаётся для каждого запуска и включается в ответ, запись платежа и событие шлюза. Он позволяет сопоставить одну операцию между пользовательским ответом, `payments_out.csv`, `payment_service.log` и `gateway.log`.

13.4. Чувствительные данные

В журналах не должны присутствовать PAN, CVV, SMS-коды, исходные токены подтверждения, пароли или секреты авторизации. В текущем коде содержимое платёжного JSON целиком не логируется.

---

14. Безопасность и ограничения

14.1. Применённые меры в рамках прототипа

- проверка обязательных полей и ограничений формата;
- разрешение только активных получателей;
- ограничение суммы и валюты;
- обнаружение PAN-подобной последовательности в назначении;
- идемпотентность и защита от повторной обработки;
- отсутствие хранения реквизитов карты в структуре запроса;
- корреляционный идентификатор для аудита;
- раздельные журналы сервиса и эмулятора.

14.2. Ограничения безопасности

Это учебный CLI, не готовая платёжная инфраструктура. В проекте нет реального эквайринга, авторизации Bearer, сетевого TLS-слоя, базы данных с транзакциями, распределённой блокировки при параллельных запросах и мониторинга доступности. CSV-журнал не обеспечивает надёжную конкурентную запись несколькими процессами. Не следует использовать прототип для обработки настоящих платежей или реальных платёжных секретов.

14.3. Поведение при параллельных запросах

Проверка и последующая запись выполняются последовательно в рамках одного процесса. Между поиском дубля и записью нет межпроцессной блокировки или транзакции. Поэтому при одновременном запуске нескольких копий возможна гонка. Для промышленной реализации потребуется транзакционное хранилище и уникальные ограничения по ключу идемпотентности и паре заказа/получателя.

---

15. Установка, запуск и эксплуатация

15.1. Подготовка

46. Установить Python 3.10+.
47. Распаковать каталог проекта, сохранив структуру `01_данные/`.
48. Проверить наличие `05_app.py` и `01_данные/recipients.csv`.
49. Не удалять и не подменять исходный `01_данные/payments.csv`.

15.2. Запуск успешного сценария

```
python3 05_app.py payment_success.json
```

15.3. Запуск отказа, валидации и таймаута

```
python3 05_app.py payment_decline.json
python3 05_app.py payment_invalid.json
python3 05_app.py payment_timeout.json
```

15.4. Проверка идемпотентности

```
python3 05_app.py payment_replay.json
python3 05_app.py payment_replay.json
```

Первый запуск создаёт запись и сохраняет ответ. Второй запуск возвращает сохранённый ответ с `replayed: true`; новая строка и новое событие шлюза не создаются.

15.5. Запуск тестов

Из корня проекта:

```
python3 -m unittest discover -s tests -v
```

Тесты работают во временных каталогах и не должны очищать рабочий `payments_out.csv` пользователя.

15.6. Создаваемые файлы

- `payments_out.csv` — накопительный журнал платежей;
- `payment_service.log` — журнал приложения;
- `gateway.log` — журнал эмулятора.

---

16. Тестирование и результаты

16.1. Обязательные сценарии кейса

ID	Сценарий	Входной пример	Проверяемый результат
T1-1	Успешная оплата	`ord_1001`, `2500.00 RUB`	`SUCCESS`, код ошибки отсутствует, запись создана
T1-2	Отказ шлюза	`ord_1002`	`DECLINED`, `INSUFFICIENT_FUNDS`
T1-3	Неверные данные	`ord_1003`, сумма `-10.00`	`AMOUNT_OUT_OF_RANGE`, HTTP-маппинг 422; платёж не создаётся
T1-4	Повтор запроса	два запроса с одинаковым ключом `idem_t1_04`	replay, один платёж, один вызов эмулятора
T1-5	Таймаут	`ord_1018`	`PENDING`, `GATEWAY_TIMEOUT`, событие TIMEOUT в журнале; повтор безопасен

16.2. Дополнительные автоматизированные проверки

В `tests/test_prototype.py` реализовано восемь тестов:

50. `test_success` — успешная операция;
51. `test_decline` — отказ эмулятора;
52. `test_validation` — отклонение отрицательной суммы;
53. `test_idempotent_replay` — точный повтор;
54. `test_timeout_and_safe_replay` — таймаут и безопасный повтор без повторного вызова эмулятора;
55. `test_idempotency_conflict` — тот же ключ с другой суммой;
56. `test_non_string_recipient_is_validation_error` — некорректный тип поля получателя;
57. `test_duplicate_order_different_key` — повтор заказа с новым ключом, проверка V-8.

Результат финального прогона в подготовленной среде: **8 тестов пройдены, ошибок нет**. Это результат проверки данной версии локального CLI в изолированных временных каталогах, а не подтверждение работы HTTP-сервера.

16.3. Критерии успешного теста

- ожидаемый статус и код ошибки совпадают с фактическими;
- некорректный запрос не вызывает эмулятор и не создаёт платёж;
- точный повтор не создаёт новую запись;
- конфликт ключа не запускает повторную обработку;
- таймаут сохраняет неопределённое состояние `PENDING`;
- `gateway.log` содержит одно событие для первого вызова, но не для replay.

---

17. Матрица реализации и дальнейшие работы

Возможность	Состояние	Комментарий
Загрузка JSON	Реализовано	Чтение одного JSON-объекта из файла
Проверка обязательных полей	Реализовано	`VALIDATION_ERROR`
Проверка `order_id`	Реализовано	Regex `^ord_[0-9]{4}$`
Проверка суммы и точности	Реализовано	Decimal + регулярное выражение
Проверка RUB	Реализовано	Другие валюты отклоняются
Проверка ACTIVE recipient	Реализовано	Загрузка CSV cp1251
Проверка purpose/PAN/CVV	Реализовано	PAN-паттерн и явная метка CVV/CVC
Дубликат заказа	Реализовано	V-8, HTTP-маппинг 409
Идемпотентность 24 часа	Реализовано	NEW / REPLAY / CONFLICT
Детерминированный эмулятор	Реализовано	4 исхода по MD5 bucket
Сохранение `response_json`	Реализовано	Replay ответа
Миграция 12 → 13 колонок	Реализовано	Для `payments_out.csv`
`payment_service.log` и `gateway.log`	Реализовано	Локальные файлы
HTTP-сервер	Не реализовано	Только OpenAPI-контракт
Bearer-аутентификация	Не реализовано	Описана в OpenAPI
`confirm_token`	Не реализовано	Документирован в `07_confirmation.md`
SMS/biometric_stub	Не реализовано	Целевые варианты подтверждения
Отдельные confirm/get/cancel	Не реализовано	Маршруты только в контракте
Проверка результата после timeout	Не реализовано	Нужен отдельный механизм сверки
Конкурентная защита CSV	Не реализовано	Нужны транзакции/блокировки

17.1. Возможности вне зафиксированной версии 1.0.0

Следующие функции сознательно не включены в зафиксированный релиз 1.0.0: HTTP-сервер на базе FastAPI, исполнение маршрутов `confirm/get/cancel`, генерация и хеширование `confirm_token`, SMS/biometric_stub, механизм сверки окончательного результата после `PENDING`, транзакционная база данных и защита от конкурентных запросов. Они остаются описанием целевого расширения и не требуются для повторного изменения принятого CLI-релиза.

---

18. Состав документации и источники

18.1. Файлы комплекта

Файл	Роль
`README.md`	Краткий запуск проекта
`02_правила.md`	Правила проверки и обработки
`03_data_dictionary.md`	Словарь данных
`04_requirements.md`	FR/NFR и сценарии
`05_app.py`	Исполняемый CLI-прототип
`06_openapi.yaml`	Целевой HTTP API
`07_confirmation.md`	Проектирование подтверждения
`08_gateway_emulator.md`	Эмулятор и его логи

Файл	Роль
`test.md`	Пять тест-сценариев для кейса
`tests/test_prototype.py`	Автоматизированные тесты
`06_приложение.md` / `06_приложение.pdf` / `06_приложение.docx`	Приложение к кейсу
`PayShield_техническая_документация.md` / `.docx`	Полная техническая документация

18.2. Исходная база

Документация опирается на первоначальное задание кейса «Платёжный сервис: безопасный платёж за услугу», инструкцию по оформлению приложения, исходные CSV-наборы, загруженные проектные документы и исправленную версию `05_app.py`, подготовленную в рамках текущей работы.

18.3. Контроль исходных данных

Исходный `payments.csv` сохранён отдельно и не изменяется прототипом. Контрольный MD5 для предоставленной копии: `39a94a80f541e44c72362304ccc3904b`. Перед окончательной сдачей рекомендуется повторно вычислить хеш непосредственно в рабочем репозитории команды.

18.4. Что требуется заполнить команде перед сдачей

- ФИО участников, фактический вклад, часы и подписи;
- фактические даты и полный состав журнала использования ИИ;
- скриншот последнего коммита в сдаваемой ветке;
- скриншоты запусков T1-1 и T1-3;
- подтверждение, какая ветка сдаётся: `feature/prototype` или предусмотренная регламентом `team_N_case_1`.

Версия 1.0.0 зафиксирована как финальная базовая версия комплекта.

Конец документа.